



PERTEMUAN 7 TRANSFORMASI GEOMETRI 3D

Kemampuan Akhir yang Diharapkan (KAD):

Mahasiswa mampu memahami dan mengimplementasikan teknik transformasi 3D yang meliputi translasi, penskalaan dan rotasi.

Pokok Bahasan:

1. Transformasi geometri 3D
2. Translasi
3. Penskalaan
4. Rotasi
5. Implementasi transformasi objek 3D dengan java.
6. Soal Latihan

7.1 Transformasi Geometri 3D

Transformasi geometri 3D merupakan pengembangan dari transformasi geometri 2D. Secara umum representasi transformasi pada 3D juga dibuat dalam bentuk matrik untuk memudahkan perhitungan. Metode transformasi objek tiga dimensi sangat berbeda dengan objek dua dimensi karena terdapat sumbu z yang ditambahkan sebagai salah satu acuan untuk memperoleh posisi koordinat baru. Sama seperti pada 2 dimensi, ada tiga transformasi dasar yang dapat dilakukan yaitu translasi, penskalaan, rotasi. Perbedaannya adalah pada objek 3 dimensi proses transformasinya dilakukan dengan mempertimbangkan koordinat yang merupakan besarnya kedalaman dari objek.

Titik hasil transformasi dapat diperoleh melalui rumus dibawah ini disebut sebagai *Affine Transformation*.

$$Q = P * M + tr$$

Dimana :

Q = (Qx, Qy, Qz) menyatakan matriks 1x3 yang berisi titik hasil transformasi.

P = (Px, Py, Pz) menyatakan matrik 1x3 yang berisi titik yang akan ditransformasi.

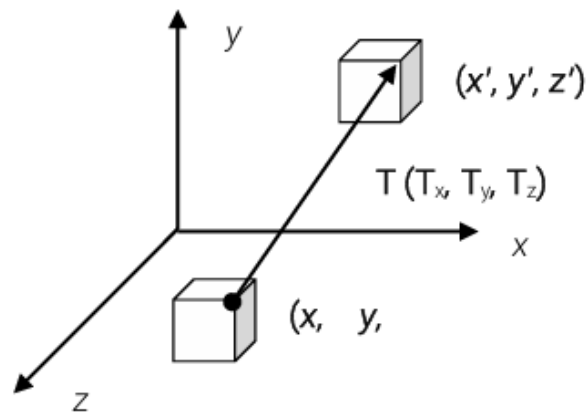
Tr = (trx, try, trz) menyatakan matrik 1x3 yang berisi banyaknya pergeseran pada sumbu x, y dan z.

M = Matriks mentransformasi berukuran 3x3 seperti di bawah ini :

$$\begin{bmatrix} m_{00} & m_{01} & m_{02} \\ m_{10} & m_{11} & m_{12} \\ m_{20} & m_{21} & m_{22} \end{bmatrix}$$

7.2 Translasi

Transformasi translasi merupakan operasi yang menyebabkan perpindahan objek tiga dimensi dari satu tempat ke tempat yang lainnya. Perubahan ini berlaku dalam arah yang sejajar dengan sumbu x, y, z. dalam operasi translasi, setiap titik pada suatu entitas yang ditranslasi bergerak dalam jarak yang sama. Pergerakan tersebut dapat berlaku dalam arah sumbu x, y, z. Untuk mentranslasikan suatu titik (x,y,z) dengan pergeseran sebesar (tx, ty, tz) menjadi titik (x",y",z") adalah:



Dalam hal ini,

$$x' = x + T_x$$

$$y' = y + T_y$$

$$z' = z + T_z$$

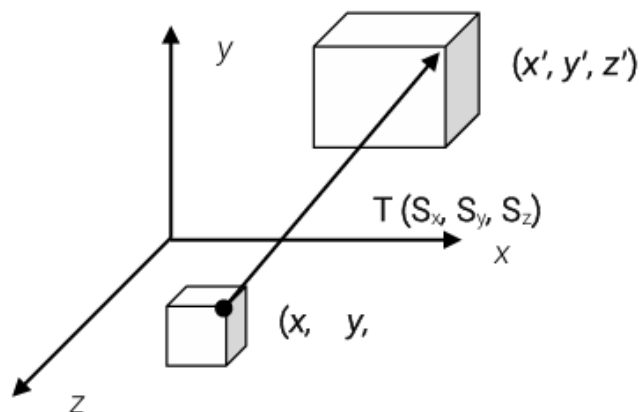
Dalam bentuk matriks:

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & T_x \\ 0 & 1 & 0 & T_y \\ 0 & 0 & 1 & T_z \\ 0 & 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

Untuk invers dari translasi dapat dilakukan dengan mengubah nilai vektor translasi menjadi negatif.

7.3 Penskalaan

Transformasi skala merupakan operasi yang menyebabkan ukuran objek berubah. Perubahan ini berlaku dalam arah yang sejajar dengan sumbu x, y, z. Dalam domain 3 dimensi, terdapat dua jenis penskalaan atau *scaling* yaitu *local scaling* dan *overall scaling* atau *global scaling*. Dalam *local scaling* penskalaan bisa dilakukan terhadap salah satu atau semua sumbu (x, y dan z). Sedangkan dalam *overall scaling* dilakukan secara seragam untuk semua sumbu. Untuk melakukan *local scaling* pada objek suatu titik (x,y,z) dengan faktor skala (sx,sy,sz) menjadi titik (x',y',z') adalah :



Dalam hal ini,

$$x' = S_x \cdot x$$

$$y' = S_y \cdot y$$

$$z' = S_z \cdot z$$

Dalam bentuk matriks:

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} S_x & 0 & 0 & 0 \\ 0 & S_y & 0 & 0 \\ 0 & 0 & S_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

Untuk invers dari skala dapat dilakukan dengan mengubah nilai faktor skala menjadi $(1/s_x, 1/s_y, 1/s_z)$.

Sedangkan formulasi transformasi untuk *global scaling* adalah:

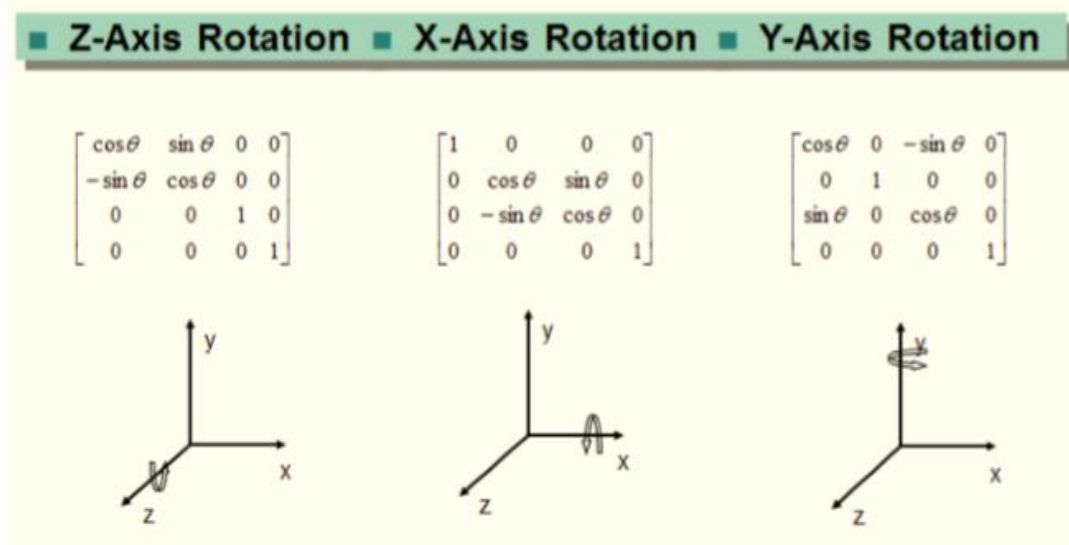
$$\begin{bmatrix} x' & y' & z' & 1 \end{bmatrix} = \begin{bmatrix} x & y & z & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & \frac{1}{S_g} \end{bmatrix}$$

7.4 Rotasi

Berbeda dengan rotasi 2 dimensi yang menggunakan titik pusat (0,0) sebagai pusat perputaran, rotasi 3 dimensi menggunakan sumbu koordinat sebagai pusat perputaran. Dengan demikian ada 3 macam rotasi yang dapat dilakukan, yaitu:

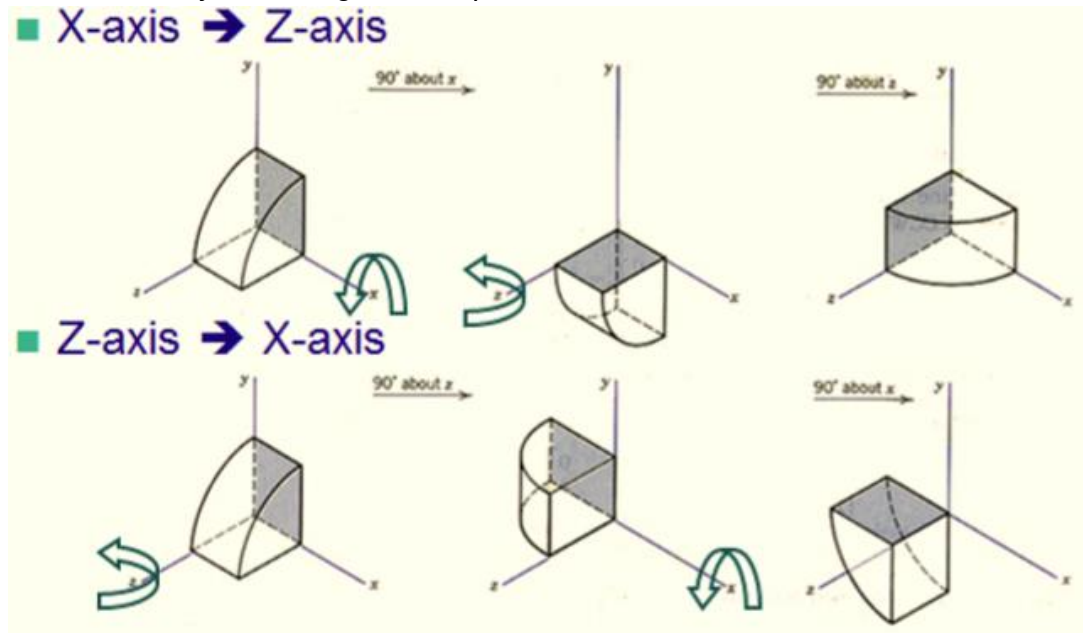
1. Rotasi sumbu x
2. Rotasi sumbu y
3. Rotasi sumbu z

Gambar 7.1 memperlihatkan bagaimana hubungan antara rotasi 3 dimensi dan sumbu rotasi.



Gambar 7.1 Rotasi dan sumbu rotasi

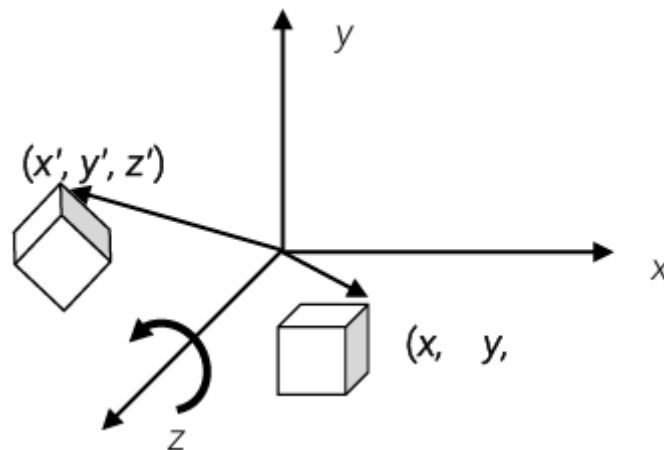
Dalam rotasi 3D perlu diperhatikan bahwa urutan proses rotasi tidaklah komutatif. Gambar berikut mengilustrasikan keadaan ini. Digambarkan bahwa hasil akhir antara urutan proses rotasi pada sumbu x dilanjutkan rotasi sumbu z TIDAK SAMA HASILNYA dengan rotasi pada sumbu z dilanjutkan dengan rotasi pada sumbu x.



Gambar 7.2 Rotasi 3D tidak berlaku sifat komutatif

Mengingat ada 3 buah sumbu rotasi maka matriks transformasi yang digunakan juga bergantung kepada sumbu putar. Adapun isi matriks transformasi sesuai dengan sumbu putar didefinisikan sebagai berikut :

Rotasi terhadap sumbu-z sebesar sudut Θ .



Dalam hal ini,

$$x' = x \cos \theta - y \sin \theta$$

$$y' = x \sin \theta + y \cos \theta$$

$$z' = z$$

Dalam bentuk matrik rotasi terhadap sumbu-z sebesar sudut Θ .

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

Dengan cara yang sama diperoleh rotasi terhadap sumbu-x sebesar sudut Θ .

$$x' = x$$

$$y' = y\cos\theta - z\sin\theta$$

$$z' = y\sin\theta + z\cos\theta$$

Dalam bentuk matrik rotasi terhadap sumbu-x sebesar sudut Θ .

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta & 0 \\ 0 & \sin\theta & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

Rotasi terhadap sumbu-y sebesar sudut Θ .

$$x' = x\cos\theta + z\sin\theta$$

$$y' = y$$

$$z' = -x\sin\theta + z\cos\theta$$

Dalam bentuk matrik rotasi terhadap sumbu-y sebesar sudut Θ .

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos\theta & 0 & \sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\theta & 0 & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

7.5 Implementasi transformasi objek 3D dengan java

A. Translasi 3D

```
// Simple 3D translation using homogeneous coordinates formula
// Formula: x' = x + Tx, y' = y + Ty, z' = z + Tz

float Tx = 50;    // translation along X
float Ty = 30;    // translation along Y
float Tz = -100;  // translation along Z

PVector[] cubeVertices;

void settings() {
  size(800, 600, P3D);
}

void setup() {
  // Define cube vertices
  cubeVertices = new PVector[] {
    new PVector(-50, -50, -50),
    new PVector( 50, -50, -50),
```

```

    new PVector( 50, 50, -50),
    new PVector(-50, 50, -50),
    new PVector(-50, -50, 50),
    new PVector( 50, -50, 50),
    new PVector( 50, 50, 50),
    new PVector(-50, 50, 50)
  };
}

void draw() {
  background(30);
  lights();

  // Move coordinate system to center of screen
  translate(width/2, height/2, 0);
  rotateY(frameCount * 0.01);
  rotateX(frameCount * 0.005);

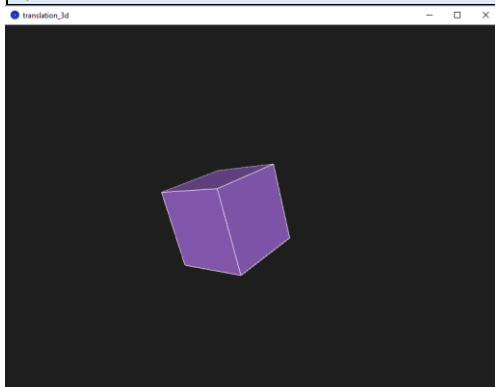
  // Draw cube after applying translation matrix
  stroke(255);
  fill(150, 100, 200);
  beginShape(QUADS);

  // Front
  vtx(0); vtx(1); vtx(2); vtx(3);
  // Back
  vtx(5); vtx(4); vtx(7); vtx(6);
  // Left
  vtx(4); vtx(0); vtx(3); vtx(7);
  // Right
  vtx(1); vtx(5); vtx(6); vtx(2);
  // Top
  vtx(4); vtx(5); vtx(1); vtx(0);
  // Bottom
  vtx(3); vtx(2); vtx(6); vtx(7);

  endShape();
}

// Apply translation formula to a vertex and send to renderer
void vtx(int i) {
  PVector v = cubeVertices[i];
  float xPrime = v.x + Tx;
  float yPrime = v.y + Ty;
  float zPrime = v.z + Tz;
  vertex(xPrime, yPrime, zPrime);
}

```



B. Penskalaan 3D

```
// Show original and scaled cube using scaling matrix formula
// Formula:  $x' = S_x * x$ ,  $y' = S_y * y$ ,  $z' = S_z * z$ 

float Sx = 1.5; // scale along X
float Sy = 1.0; // scale along Y
float Sz = 0.5; // scale along Z

PVector[] cubeVertices;

void settings() {
    size(900, 600, P3D);
}

void setup() {
    // Define cube vertices
    cubeVertices = new PVector[] {
        new PVector(-50, -50, -50),
        new PVector( 50, -50, -50),
        new PVector( 50,  50, -50),
        new PVector(-50,  50, -50),
        new PVector(-50, -50,  50),
        new PVector( 50, -50,  50),
        new PVector( 50,  50,  50),
        new PVector(-50,  50,  50)
    };
}

void draw() {
    background(30);
    lights();

    // Move origin to center
    translate(width/2, height/2, 0);
    //rotateY(frameCount * 0.01);
    //rotateX(frameCount * 0.005);

    // Draw original cube (left side)
    pushMatrix();
    translate(-150, 0, 0);
    drawCube(false);
    popMatrix();

    // Draw scaled cube (right side)
    pushMatrix();
    translate(150, 0, 0);
    drawCube(true);
    popMatrix();

    // Labels
    fill(255);
    textAlign(CENTER);
    text("Original", -150, 150);
    text("Scaled", 150, 150);
}

void drawCube(boolean scaled) {
    stroke(255);
```

```

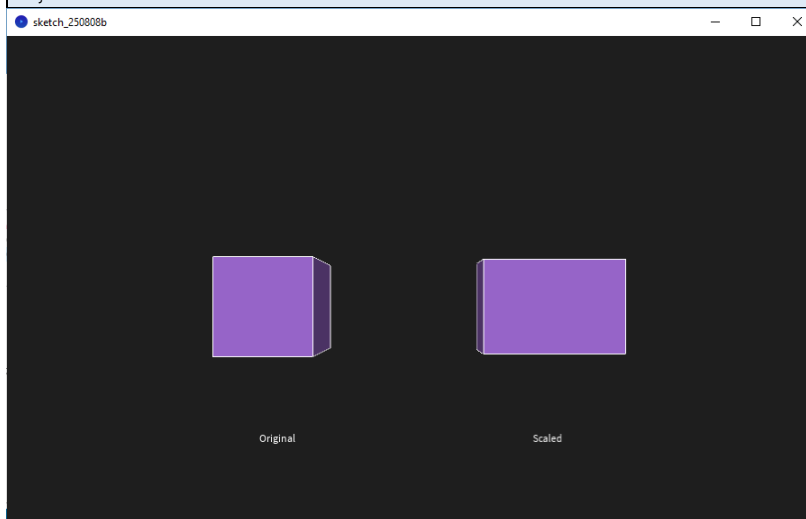
fill(150, 100, 200);
beginShape(QUADS);

// Front
vtx(0, scaled); vtx(1, scaled); vtx(2, scaled); vtx(3, scaled);
// Back
vtx(5, scaled); vtx(4, scaled); vtx(7, scaled); vtx(6, scaled);
// Left
vtx(4, scaled); vtx(0, scaled); vtx(3, scaled); vtx(7, scaled);
// Right
vtx(1, scaled); vtx(5, scaled); vtx(6, scaled); vtx(2, scaled);
// Top
vtx(4, scaled); vtx(5, scaled); vtx(1, scaled); vtx(0, scaled);
// Bottom
vtx(3, scaled); vtx(2, scaled); vtx(6, scaled); vtx(7, scaled);

endShape();
}

// Apply scaling formula if scaled=true
void vtx(int i, boolean scaled) {
  PVector v = cubeVertices[i];
  float xPrime = scaled ? Sx * v.x : v.x;
  float yPrime = scaled ? Sy * v.y : v.y;
  float zPrime = scaled ? Sz * v.z : v.z;
  vertex(xPrime, yPrime, zPrime);
}

```



C. Rotasi 3D

```

// 3D rotation around Z-axis using matrix formula
// Formula:
// x' = x*cosθ - y*sinθ
// y' = x*sinθ + y*cosθ
// z' = z

float angle = 0; // rotation angle in radians

PVector[] cubeVertices;

void settings() {
  size(800, 600, P3D);
}

```



```
void setup() {
    // Define cube vertices
    cubeVertices = new PVector[] {
        new PVector(-50, -50, -50),
        new PVector( 50, -50, -50),
        new PVector( 50,  50, -50),
        new PVector(-50,  50, -50),
        new PVector(-50, -50,  50),
        new PVector( 50, -50,  50),
        new PVector( 50,  50,  50),
        new PVector(-50,  50,  50)
    };
}

void draw() {
    background(30);
    lights();

    translate(width/2, height/2, 0);

    stroke(255);
    fill(150, 100, 200);
    beginShape(QUADS);

    // Front
    vtx(0); vtx(1); vtx(2); vtx(3);
    // Back
    vtx(5); vtx(4); vtx(7); vtx(6);
    // Left
    vtx(4); vtx(0); vtx(3); vtx(7);
    // Right
    vtx(1); vtx(5); vtx(6); vtx(2);
    // Top
    vtx(4); vtx(5); vtx(1); vtx(0);
    // Bottom
    vtx(3); vtx(2); vtx(6); vtx(7);

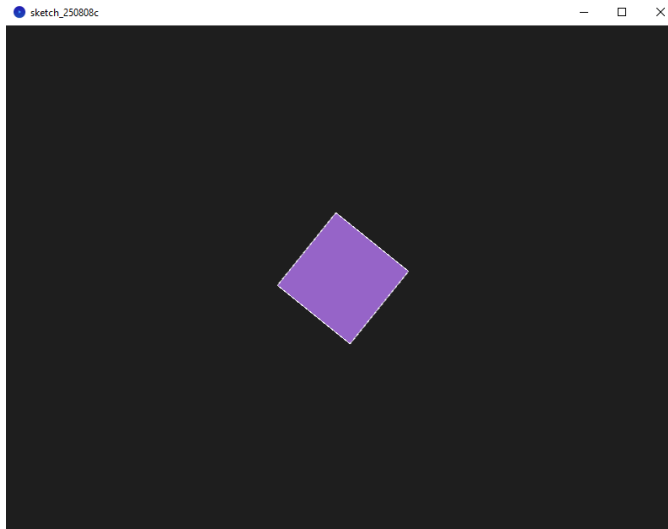
    endShape();

    // Increase angle
    angle += 0.01;
}

// Apply rotation matrix for Z-axis
void vtx(int i) {
    PVector v = cubeVertices[i];
    float cosA = cos(angle);
    float sinA = sin(angle);

    float xPrime = v.x * cosA - v.y * sinA;
    float yPrime = v.x * sinA + v.y * cosA;
    float zPrime = v.z;

    vertex(xPrime, yPrime, zPrime);
}
```



```
// 3D rotation around X-axis using matrix formula
// Formula:
// x' = x
// y' = y*cosθ - z*sinθ
// z' = y*sinθ + z*cosθ

float angle = 0; // rotation angle in radians

PVector[] cubeVertices;

void settings() {
  size(800, 600, P3D);
}

void setup() {
  // Define cube vertices (size 100)
  cubeVertices = new PVector[] {
    new PVector(-50, -50, -50),
    new PVector( 50, -50, -50),
    new PVector( 50,  50, -50),
    new PVector(-50,  50, -50),
    new PVector(-50, -50,  50),
    new PVector( 50, -50,  50),
    new PVector( 50,  50,  50),
    new PVector(-50,  50,  50)
  };
}

void draw() {
  background(30);
  lights();

  translate(width/2, height/2, 0);

  stroke(255);
  fill(150, 100, 200);
  beginShape(QUADS);

  // Front
  vtx(0); vtx(1); vtx(2); vtx(3);
  // Back
```

```

vtx(5); vtx(4); vtx(7); vtx(6);
// Left
vtx(4); vtx(0); vtx(3); vtx(7);
// Right
vtx(1); vtx(5); vtx(6); vtx(2);
// Top
vtx(4); vtx(5); vtx(1); vtx(0);
// Bottom
vtx(3); vtx(2); vtx(6); vtx(7);

endShape();

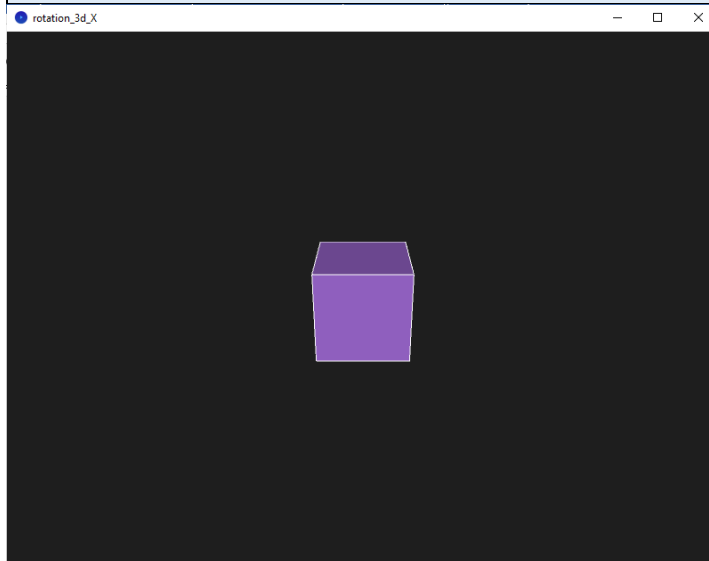
// Increase angle for animation
angle += 0.01;
}

// Apply rotation matrix for X-axis
void vtx(int i) {
  PVector v = cubeVertices[i];
  float cosA = cos(angle);
  float sinA = sin(angle);

  float xPrime = v.x;
  float yPrime = v.y * cosA - v.z * sinA;
  float zPrime = v.y * sinA + v.z * cosA;

  vertex(xPrime, yPrime, zPrime);
}

```



```

// 3D rotation around Y-axis using matrix formula
// Formula:
// x' = x*cosθ + z*sinθ
// y' = y
// z' = -x*sinθ + z*cosθ

float angle = 0; // rotation angle in radians

PVector[] cubeVertices;

void settings() {
  size(800, 600, P3D);
}

```

```

}

void setup() {
  // Define cube vertices (size 100)
  cubeVertices = new PVector[] {
    new PVector(-50, -50, -50),
    new PVector( 50, -50, -50),
    new PVector( 50,  50, -50),
    new PVector(-50,  50, -50),
    new PVector(-50, -50,  50),
    new PVector( 50, -50,  50),
    new PVector( 50,  50,  50),
    new PVector(-50,  50,  50)
  };
}

void draw() {
  background(30);
  lights();

  translate(width/2, height/2, 0);

  stroke(255);
  fill(150, 100, 200);
  beginShape(QUADS);

  // Front
  vtx(0); vtx(1); vtx(2); vtx(3);
  // Back
  vtx(5); vtx(4); vtx(7); vtx(6);
  // Left
  vtx(4); vtx(0); vtx(3); vtx(7);
  // Right
  vtx(1); vtx(5); vtx(6); vtx(2);
  // Top
  vtx(4); vtx(5); vtx(1); vtx(0);
  // Bottom
  vtx(3); vtx(2); vtx(6); vtx(7);

  endShape();

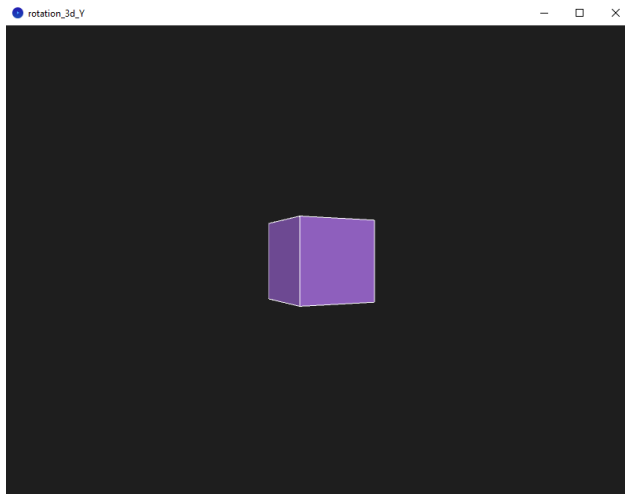
  // Increase angle for animation
  angle += 0.01;
}

// Apply rotation matrix for Y-axis
void vtx(int i) {
  PVector v = cubeVertices[i];
  float cosA = cos(angle);
  float sinA = sin(angle);

  float xPrime = v.x * cosA + v.z * sinA;
  float yPrime = v.y;
  float zPrime = -v.x * sinA + v.z * cosA;

  vertex(xPrime, yPrime, zPrime);
}

```



7.6 Soal Latihan

Latihan Penskalaan

Diketahui sebuah objek P dengan koordinat sebagai berikut : $\{(0,0,1,1), (2,0,1,1), (2,3,1,1), (0,3,1,1), (0,0,0,1), (2,0,0,1), (2,3,0,1), (0,3,0,1)\}$.

1. Gambarkan objek tersebut !
2. Lakukan local scaling terhadap objek P dengan faktor skala $xyz=\{1/2, 1/3 \text{ dan } 1\}$.
 - a. Tentukan koordinat baru
 - b. Gambarkan hasilnya
3. Lakukan overall scaling terhadap objek asli dengan faktor 2.
 - a. Tentukan koordinat baru
 - b. Gambarkan hasilnya

Latihan Rotasi

Diketahui sebuah objek Q dengan koordinat sebagai berikut : $\{(0,0,1,1), (3,0,1,1), (3,2,1,1), (0,2,1,1), (0,0,0,1), (3,0,0,1), (3,2,0,1), (0,2,0,1)\}$.

1. Gambarkan objek tersebut !
 - a. Lakukan rotasi terhadap Q sebesar $\theta = -90^\circ$ pada x
 - b. Tentukan koordinat baru
 - c. Gambarkan hasilnya
2. Lakukan rotasi terhadap objek Q sebesar $\phi = 90^\circ$ pada sumbu y
 - a. Tentukan koordinat baru
 - b. Gambarkan hasilnya

Daftar Pustaka

1. Andono, Pulung Nurtantio., Sutojo, T. 2016. Konsep Grafika Komputer. Yogyakarta: CV Andi Offset.
2. Grafika Komputer untuk Mahasiswa dan Umum. Universitas Negeri Malang.
3. Maliki, Irfan. 2006. Grafika Komputer. Jakarta Selatan: Mediatek.
4. Siahaan, Vivian., Sianipar, Rismon H., Java untuk Grafika Komputer dan Animasi, SPARTA, 2018.
5. Suhartono, Vincent. 2016. Pengantar Computer Graphics Algoritma Dasar. Semarang: CV Mulia Jaya.